Software Entwicklungsplan

Projekttitel	Feels Like
Erstellt am	10. Oktober 2025
Zuletzt aktualisiert von	S Sophie Lazarjan
Zuletzt aktualisiert um	19. Februar 2026 11:14
Überprüfer/-innen	J Julian Stengele A Atu I Isabella Schwarz J Joey S Sophie Lazarjan
Erstellt von	J Julian Stengele
Kategorie	Strategiedokument

Versionsverwaltung

Version	Datum	Änderungen
V 1.0.0	10. Oktober 2025	Initialversion
V 1.0.1	10. Oktober 2025	Anpassung Teamzusammensetzung/Rollen
V 1.1.0	17. Oktober 2025	Aktivitätsdiagramm Vorgehensmodell
V 2.0.0	17. Oktober 2025	Qualitätssicherung/-maßnahmen
V 3.0.0	17. November 2025	Aufwandsschätzung mittels ABCD-Methode
V 3.1.0	9. Januar 2026	Anpassung Aufwandsschätzung: ABCD-Methode → Function Point Analyse

Vorgehensmodell

SCRUM:

- Sprint Planning: Das Team wählt Aufgaben aus dem Product Backlog aus und plant, was im nächsten Sprint umgesetzt wird.
- Daily Scrum: Kurzes, tägliches Meeting (max. 15 Min.) für das Entwicklungsteam, um den Fortschritt zu besprechen und Hindernisse schnell zu erkennen
- Sprint Review: Das Team präsentiert die fertigen Funktionen, um direktes Feedback für die weitere Entwicklung zu erhalten.
- Retrospective: Das Team bespricht, was im letzten Sprint gut und was schlecht lief, um den eigenen Arbeitsprozess kontinuierlich zu verbessern.

▼ **Aktivitätsdiagramm**

Teamzusammensetzung und Rollen

▼ **Product Owner**

Der Product Owner vertritt die Interessen der Stakeholder und ist für den wirtschaftlichen Erfolg des Produkts verantwortlich. Er definiert die Anforderungen im Product Backlog und priorisiert diese, um den maximalen Wert zu schaffen.

- **Aufgaben:** Kommunikation mit Stakeholdern, Definition und Priorisierung der Produktanforderungen, Abnahme der umgesetzten Features.

▼ **SCRUM Master**

Der Scrum Master ist dafür verantwortlich, dass das Scrum-Framework eingehalten wird. Er moderiert die Meetings, beseitigt Hindernisse (Impediments) und sorgt dafür, dass das Team produktiv und ungestört arbeiten kann.

- **Aufgaben:** Organisation der Scrum-Meetings, Beseitigung von Hindernissen, Förderung der Team-Zusammenarbeit und des kontinuierlichen Verbesserungsprozesses.

▼ **Development Team**

Das Development Team ist ein selbstorganisiertes, interdisziplinäres Team, das gemeinsam für die Entwicklung und Qualität des Produkts verantwortlich ist. Es plant die Umsetzung der Anforderungen in den Sprints und liefert am Ende jedes Sprints ein fertiges Produktinkrement.

- **Gemeinsame Verantwortlichkeiten des Teams:**
  - **Qualitätssicherung:** Entwicklung von Qualitätsstandards, Definition von Testverfahren und Durchführung von Reviews. Jedes Teammitglied ist für die Qualität verantwortlich.
  - **Risikomanagement:** Identifikation und Bewertung potenzieller Risiken sowie die Entwicklung von Gegenmaßnahmen.
  - **Koordination & Planung:** Gemeinsame Planung der Sprints, Aufteilung der Aufgaben und Sicherstellung eines reibungslosen Informationsflusses.

Person	Rolle(n)
Atussa Mehrawari	Product Owner, Mitglied des Development Teams
Isabella Schwarz	Mitglied des Development Teams
Sophie Lazarjan	Mitglied des Development Teams
Joey Stöckle	Mitglied des Development Teams
Julian Stengele	SCRUM-Master, Mitglied des Development Teams

Aufgaben und Verantwortlichkeiten

▼ Datenbank- und Backend Entwicklung

Aufgaben:

- Entwurf und Implementierung der Datenbankstruktur (z. B. Tabellen, Relationen, Datentypen)
- Erstellung von SQL-Statements und Stored Procedures
- Aufbau einer sicheren Schnittstelle zwischen Datenbank und Anwendung
- Gewährleistung von Datenintegrität, Konsistenz und Performance
- Testen und Dokumentieren der Datenbankfunktionen

Ziel:

Ein stabiles, effizientes und skalierbares Backend, das alle Daten zuverlässig verwaltet.

▼ Frontend Entwicklung/UI-Design

Aufgaben:

- Gestaltung und Umsetzung der Benutzeroberfläche
- Sicherstellung einer intuitiven und benutzerfreundlichen Bedienung
- Implementierung von Frontend-Funktionen (z. B. Eingabeformulare, Datenvisualisierung)
- Abstimmung mit dem Backend-Team zur Schnittstellenintegration
- Usability-Tests und Feedback-Auswertung

Ziel:

Ein ansprechendes, nutzerfreundliches Interface, das den Anforderungen der Athlet:innen entspricht.

▼ Anforderungsanalyse und Konzeption

Aufgaben:

- Aufnahme und Dokumentation der fachlichen Anforderungen
- Erstellung von Use Cases und Prozessabläufen
- Abstimmung mit Stakeholdern zur Definition des Funktionsumfangs
- Erstellung von Mockups oder Konzeptskizzen
- Unterstützung bei der Priorisierung von Features

Ziel:

Sicherstellung, dass das Endprodukt den Bedürfnissen von Athlet:innen und Unternehmen entspricht.

▼ Testing und Qualitätssicherung

Aufgaben:

- Erstellung von Testplänen und Testfällen
- Durchführung von Funktionstests, Integrationstests und Systemtests
- Dokumentation und Nachverfolgung von Fehlern
- Überprüfung der Performance und Zuverlässigkeit

- Sicherstellung der Datenkonsistenz und Einhaltung der Anforderungen

Ziel:

Ein fehlerfreies, stabiles und performantes Endprodukt.

▼ Dokumentation

Aufgaben:

- Erstellung und Pflege der Projektdokumentation
- Zusammenfassung technischer und fachlicher Ergebnisse
- Vorbereitung von Präsentationen und Demos für Stakeholder
- Nachhalten von Änderungen im Changelog
- Unterstützung bei der Endabgabe und Bewertung

Ziel:

Nachvollziehbare, professionelle Dokumentation des gesamten Projektverlaufs.

Meilensteine

Nicht vorhanden → SCRUM-Vorgehensmodell

Zugehörige Dokumente

- Projektskizze
- Software Entwicklungsplan
- Meeting Protokolle
- Sprint Review Protokoll
- Change Log

Technologien

- Notion: Dokumentation und Sammlung aller dazugehörigen Unterlagen
- IDE: VS Code/IntelliJ
- Programmiersprachen: Javascript
- Frameworks & Laufzeitumgebungen: React, Node.js, Express.js
- API-Schnittstelle: REST-APIs
- Datenbank: SQL (genauere Definition später notwendig)
- Versionsverwaltung: Git, GitHub
- Hosting-Provider: AWS, Google Cloud, Azure

Qualitätsmaßnahmen

Qualitätsziel	Beschreibung	Messkriterium
Zuverlässigkeit	Bei gleichzeitiger Benutzung soll die Seite jeder Zeit verfügbar sein, damit Athlet/innen ihre Trainigsdaten immer	Serververfügbarkeit ≥ 99%, keine Datenverluste bei gleichzeitigem Zugriff

	abrufen und einfügen können	
Wartbarkeit	effiziente Anpassung und Erweiterung, damit schnell auf Fehler im Code oder in der Funktionalität behoben werden kann	kleine Änderungen (Fehlerbehebung und neue/zusätzliche Features) innerhalb von 2-4 Tagen umsetzbar; größere Änderungen: monatliche Updates
Performance	Die Seite soll schnell laden und keine unnötige Verzögerungen beinhalten. Datenbankabfragen werden optimiert um schnelle Reaktionszeiten zu gewährleisten.	Seite lädt in unter 3 Sekunden Ranglistenaktualisierung/Trainingsdatenaktualiserung in unter 10 Sekunden nach neuer Eingabe
Benutzerfreundlichkeit	Intuitive Bedienung für Sportler:innen & Trainer:innen: Die Seite soll verständlich und einfach zu bedienen sein. Komplizierte Funktionen sollten erklärt werden. Zur Unterstützung des visuellen Verständnisses können Icons und Bilder verwendet werden.	mindestens eine Zufriedenheit von 85% bei einer Nutzerumfrage in den ersten 3 Monaten
Datenqualität	Fehlerfreiheit, Vollständigkeit, Korrektheit, Konsistenz der Traningsdaten; Erfüllung von ISO 8000:	Fehlerrate bei Trainingsdaten ≤ 1 % (Validierung verhindert unplausible Eingaben)
Datenschutz	DSGVO -Art. 4 Gesundheistdaten (besondere Kategorie personenberzogener Daten) -Art. 9 Datenverarbeitung verboten ohne ausdrückliche Zustimmung / rechtliche Grundlage -Art. 32 Verpflichtung zur angemessenen technischen und Organisationen für Sicherheit -ISO 27701 (Datenschutz)(optional)	-Einwilligungen, 100% der sensiblen Daten werden verschlüsselt gespeichert, Zugriff nur für authentifizierte Nutzer -ISO 27701 Zertifikat
Skalierbarkeit	Vermeidung von Engpässen bei steigender Anzahl Athlet:innen oder erhöhter Nutzung in Wettkampfphasen	Vervielfältigung (10 fache) von Nutzeranzahlen sollen mit Verlust von < 10% bearbeitet werden

Qualitätssicherung und Tests

Teststufe	Ziel	Beschreibung	Testwerkzeuge und Frameworks	Zeitlicher Durchlauf

Unit Test	Überprüfung einzelner Komponenten oder Funktionen	Überprüfung zentraler Backend Funktionen	Progammiersprachenabhängige Tests (Jest)	Bei jedem Commit/Pull-Request
Integrationstest	Sicherstellen dass die einzelnen Module korrekt miteinander interagieren	Überprüfung der Schnittstelle zwischen Frontend und Backend beim Eintragen neuer Trainingsdaten	Postman (API Tests)	
Systemtest	Test der gesamten Anwendung in einer produktionsähnlichen Umgebung	Test aller Hauptfunktionen (Login, Datenerfassung, Ranking Anzeige) in Kombination	End-to-End (E2E) Test-Framework (Simulation vom Benutzerverhalten im Browser)	Nach Änderungen und neuen Updates
Abnahmetest	Validierung durch Auftraggeber und Endnutzer	Testdurchlauf mit echten Athleten und Feedback zur Benutzerfreundlichkeit und Datenkorrektheit	Manuelle tests, Feedbackformular	Vor dem ersten Launch bzw. vor großen Updates
Sicherheitstests	Prüfung der IT-Sicherheit im Allgemeinen	Überprüfung der Standfestigkeit gegen Angriffen, zufälligen Eingaben und Berechtigungen (Angriffssimulationen, Zugriffstests)	Burp Suite, (Burp Intruder)	Nach größeren Architekturänderungen und Updates; unabhängig davon halbjährig

Qualitätsmetriken und Kontrolle

- ISO 27001 Internationaler Standard für IT-Sicherheit
- User Experience / Fehlerberichte von Anwendern

Testabdeckung	70-80%	70-80% des Codes getestet → hohe Sicherheit
Bugs pro KLOC	6	6 Bugs pro 1000 Zeilen Code
Lines of Code	-Methoden ≤ 20-30 Zeilen -Klassen ≤ 300-500 Zeilen	-Indikator: Zu lange Mehoden/Klassen → hohe Wahrscheinlichkeit für komplexe Logik
Tiefe der Verschachtelung	Tiefe < 4	Misst wie oft if, for, while, switch verschachtelt sind. -Schwer zu verstehen bei hoher Tiefe
Anzahl der Parameter	Parameter < 4	Methoden mit vielen Mehoden sind schwer zu verstehen

Style Guide

- Standards

  Als Standard wird mindestens **ECMAScript 2015 (ES6)** empfohlen

- Imports

  Die Importe sollen nach Gruppen Sortiert werden. Innerhalt dieser Gruppen sollen diese zusätzlich Alphabetisch sortiert sein. Die Importgruppen werden durch leere Zeilen voneinander getrennt. Es sollten keine ungenutzten Imports vorhanden sein,.

  - Node.js / JavaScript Built-ins

    - Drittanbieter-Libraries
    - Interne Projektmodule
    - Side-Effect Imports (z. B. CSS)
- Klassen, Funktionen, Variblen
    - **Klassen**namen sollten im **PascalCase** gehalten werden.
    - **Funktion**snamen sollten im **camelCase** gehalten werden.
    - **Variablen**namen sollten im **camelCase** gehalten werden.
- Getter & Setter
    - Getter & Setter sollten sollten gleich wie die Variable heißen. Ausnahme sind Getter für boolsche Variablen. Diese sollten is<Variablenname> heißen.

▼ **Aufwandsschätzung (ABCD-Methode) → ALT**

Schätzprozess

Um eine nachvollziehbare Aufwandsschätzung zu gewährleisten, nutzen wir die **ABCD-Methode**. Als Basis für die Schätzung dienen die 8 definierten Use Cases (`UC-01` bis `UC-08`) aus dem Pflichtenheft.

Schätzung

Die Bewertung der Use Cases erfolgt im Rahmen einer **Schätzkonferenz** im gesamten Projektteam. Für jeden Use Case werden die vier Faktoren (A: Wiederverwendung, B: Funktionalität, C: Schnittstellen, D: Kritikalität) diskutiert und im Konsens festgelegt. Der finale Aufwandspunkt (AP) für jeden Use Case ergibt sich aus der Multiplikation `A * B * C * D`.

Integration Risikomanagement

Das Risikomanagement ist durch den **Faktor D (Kritikalität)** fest in den Schätzprozess integriert. Use Cases, die für den Projekterfolg kritisch sind (z.B. `UC-01: Kontoverwaltung`), erhalten einen höheren D-Wert, was den Test- und Qualitätsaufwand direkt in den geschätzten Aufwandspunkten abbildet.

Die folgende Reihenfolge der Use Cases entspricht der Reihenfolge der Bearbeitung der Einschätzung:

▼ UC-01

Beschreibung	**A (Wiederverwendung - 1-3)**	**B (Funktionalität - 1-8)**	**C (Schnittstellen - 1-3)**	**D (Kritikalität - 1-2)**	**Gesamt AP**
UC 01 - Kontoverwaltung	3	5	3	2	90
Begründung der Schätzung	Muss neu erstellt werden	Mittelgroßer Baustein der Entwicklung	Zentrale Komponente des Projekts	Kritische Komponente	

▼ UC-02

Beschreibung	A (Wiederverwendung - 1-3)	B (Funktionalität - 1-8)	C (Schnittstellen - 1-3)	D (Kritikalität - 1-2)	Gesamt AP
UC-02 Athletenverwaltung & Einladung	1,9	3,5	3	2	39,9
Begründung der Schätzung	Wiederverwendbarkeit der bestehenden Kontoverwaltung	Relativ gesehen kleiner als Kontoverwaltung	Abhängigkeit von der Kontoverwaltung	Für Feels Like Organic Team sonst nutzlos	

▼ UC-04

Beschreibung	**A (Wiederverwendung - 1-3)**	**B (Funktionalität - 1-8)**	**C (Schnittstellen - 1-3)**	**D (Kritikalität - 1-2)**	**Gesamt AP**
UC 04 - Trainingseinheiten verwalten	3	3	2	1,5	27
Begründung der Schätzung	Neuer Baustein	Arbeit mit Daten	Normale Abhängigkeiten von anderen Teilaufgaben	Wichtige Funktionalität für Athleten	

▼ UC-03

Beschreibung	**A (Wiederverwendung - 1-3)**	**B (Funktionalität - 1-8)**	**C (Schnittstellen - 1-3)**	**D (Kritikalität - 1-2)**	**Gesamt AP**
UC 03 - Wettbewerbsdaten verwalten	1,6	2	2	1	6,4
Begründung der Schätzung	Wiederverwendung von UC-04	-	Normale Abhängigkeiten	Keine hohe Kritikalität	

▼ UC-05

Beschreibung	**A (Wiederverwendung - 1-3)**	**B (Funktionalität - 1-8)**	**C (Schnittstellen - 1-3)**	**D (Kritikalität - 1-2)**	**Gesamt AP**
UC-05 Persönliche Statistiken anzeigen	1,9	3	2	1	11,4
Begründung der Schätzung	Kein hoher Grad der Wiederverwendung		Abhängigkeit von Daten		

▼ UC-06

Beschreibung	**A (Wiederverwendung - 1-3)**	**B (Funktionalität - 1-8)**	**C (Schnittstellen - 1-3)**	**D (Kritikalität - 1-2)**	**Gesamt AP**
UC 06 - Ranking-Filter durchführen	1,9	3,5	3	1,5	29,9
Begründung der Schätzung		Statistiken aller Athleten	Zugriff auf alle Athleten-Daten		

▼ UC-07

Beschreibung	A (Wiederverwendung -1-3)	B (Funktionalität -1-8)	C (Schnittstellen -1-3)	D (Kritikalität -1-2)	Gesamt AP
UC-07: Passwort über E-Mail zurücksetzen	1,6	2	2	1,5	9,6
Begründung der Schätzung	Mailsystem von Kontoverwaltung kann wiederverwendet werden		Mailsystem, Kontoverwaltung	System bleibt nutzbar, Zufriedenheit eingeschränkt	

▼ UC-08

Beschreibung	A (Wiederverwendung - 1-3)	B (Funktionalität - 1-8)	C (Schnittstellen - 1-3)	D (Kritikalität - 1-2)	Gesamt AP
UC 08 - Trainingsdaten exportieren	3	3,4	2	1	20,4
Begründung der Schätzung	Neue Funktionalität		Abhängigkeit von Daten	Keinen großen Einfluss auf die Nutzbarkeit	

Gesamt AP über alle UC: 234,6

Aufwandsschätzung (Function Point Analyse)

Methode

Zur Ermittlung des voraussichtlichen Aufwands wird die **Function Point Analyse (FPA)** nach IFPUG-Standard verwendet. Diese Methode bewertet die funktionale Größe des Softwaresystems basierend auf den definierten Anforderungen und dem Datenmodell, unabhängig von der technischen Implementierung.

Identifizierung der Funktionen

Die Schätzung basiert auf den im Pflichtenheft definierten Use Cases und dem im Grobdesign erstellten Datenmodell.

Datenbestand (Data Functions)

Hier werden die internen logischen Dateigruppen (ILF) und externen Referenzdateien (EIF) bewertet. Basierend auf unserem Analyseklassenmodell identifizieren wir folgende Entitäten:

Typ	Beschreibung / Entität	Komplexität	Begründung (DETs/RETs)	Function Points
ILF	**Athlet / Benutzerkonto**	Low	Wenige Attribute (Name, Email, PW), 1 Record Type	**7**
ILF	**Trainingseinheit**	Low	Attribute (Datum, Dauer, Distanz, Sportart), 1 Record Type	**7**
ILF	**Wettbewerb**	Low	Attribute (Datum, Zeit, Platzierung, Status), 1 Record Type	**7**
ILF	**Filter-Konfiguration**	Low	Speicherung von Filtereinstellungen (Statistik/Ranking)	**7**
Summe				**28 FP**

Die Entitäten werden als "Low" eingestuft, da sie eine überschaubare Anzahl an Attributen haben.

Transaktionsfunktionen (Transactional Functions)

Hier werden die Eingaben (EI), Ausgaben (EO) und Abfragen (EQ) aus den Use Cases bewertet.

Use Case	Prozess / Funktion	Typ	Komplexität	FP
UC-01	Registrierung durchführen	EI	Low	3
	Anmeldung (Login prüfen)	EQ	Low	3
	Profil/Passwort ändern	EI	Low	3
UC-02	Athlet anlegen (durch Admin)	EI	Low	3
	Einladungs-Email versenden	EO	Low	4
	Athletenliste anzeigen	EQ	Low	3
UC-03	Wettbewerb erfassen	EI	Low	3
	Wettbewerb ändern	EI	Low	3

	Wettbewerb löschen	EI	Low	3
	Wettbewerbsdetails einsehen	EQ	Low	3
UC-04	Trainingseinheit erfassen	EI	Low	3
	Trainingseinheit ändern	EI	Low	3
	Trainingseinheit löschen	EI	Low	3
UC-05	Statistik berechnen & anzeigen	EO	Average	5
UC-06	Ranking berechnen (komplexe Logik)	EO	High	7
UC-07	Passwort-Reset anfordern	EI	Low	3
	Neues Passwort setzen	EI	Low	3
UC-08	Daten exportieren (CSV Generierung)	EO	Low	4
Summe				**62 FP**

(Legende: EI = External Input, EO = External Output, EQ = External Inquiry. Komplexität Average/High wurde gewählt, wo Berechnungen wie Statistiken oder Rankings stattfinden.)

Unadjusted Function Points (UFP)
Die Summe der Funktionspunkte ohne Einflussfaktoren beträgt:
UFP = Datenfunktionen (28) + Transaktionsfunktionen (59) = **87 FP**

Einflussfaktoren (General System Characteristics)
Zur Berechnung der Adjusted Function Points (AFP) bewerten wir die 14 allgemeinen Systemcharakteristika auf einer Skala von 0 (kein Einfluss) bis 5 (starker Einfluss).

Nr.	Einflussfaktor	Bewertung (0-5)	Begründung
1	Datenkommunikation	4	Webanwendung, ständige Online-Kommunikation nötig.
2	Verteilte Datenverarbeitung	0	Zentrale Datenbank, keine verteilten Systeme.
3	Performance-Anforderungen	4	Antwortzeiten < 3s gefordert
4	Starke Nutzung der Konfiguration	0	Keine komplexe Hardware-Konfiguration.
5	Transaktionsrate	2	Erwartet moderat, Spitzenlast bei Wettkämpfen.
6	Online-Datenerfassung	5	Hauptzweck der App (Training/Wettbewerb erfassen).
7	Endbenutzer-Effizienz	5	Hohe Usability gefordert (Mobile Nutzung).
8	Online-Update (Datenbestand)	5	Daten werden sofort in DB geschrieben (CRUD).
9	Komplexität der Verarbeitung	3	Ranking- und Statistik-Algorithmen.
10	Wiederverwendbarkeit	1	Code ist spezifisch für dieses Projekt.
11	Installationsfreundlichkeit	2	Standard Web-Deployment.
12	Operative Einfachheit	4	Einfache Bedienung für Athleten essenziell.
13	Mehrfachinstallationen	0	Nur eine zentrale Instanz.
14	Anpassungsfreundlichkeit	3	Modularer Aufbau für künftige Erweiterungen.
Summe	**Total Degree of Influence (TDI)**	**38**	

Ergebnis: Adjusted Function Points (AFP)
Der Value Adjustment Factor (VAF) berechnet sich wie folgt:
VAF = (TDI ∗ 0.01) + 0.65
VAF = (38 ∗ 0.01) + 0.65 = 1.03

Daraus ergeben sich die finalen Function Points:
AFP = UFP ∗ VAF

AFP = 90 ∗ 1.03 = 92,7

Geschätzter Gesamtumfang: ~93 Function Points

Das Risikomanagement ist in dieser Schätzung durch die Bewertung der Einflussfaktoren (GSC) integriert, welche die technische Komplexität und Qualitätsanforderungen berücksichtigen und den Aufwand entsprechend nach oben korrigieren.